

HACKATHON WORK BREAKDOWN STRUCTURE (WBS)

Problem #5: Equipment Rental Decision Agent (Chatbot Edition)

Event: Cymonic Campus Recruitment - Agentic Workforce Hackathon
Time Allotted: 5 Hours | Team Size: 4 Members
Core Philosophy: Dynamic Agentic Reasoning via Natural Language Parsing

1. The Core Strategy (Team-Wide Understanding)

We are building a Chatbot Interface (Streamlit) that ingests unstructured contractor inquiries (e.g., *"Need a crane urgently tomorrow"*).
The agent will parse the text, calculate a Qualification Score (0-100) across 4 weighted gates (Availability, License, Lead Time, Contractor Tier), and output one of three actions:

- ✔ Auto-Quote (Score ≥ 80)
- ⚠ Request More Info (Score 50–79)
- 🔔 Manual Review (Score < 50)

2. Individual Role Assignments & Deliverables

Team Member	Role / Focus	Time Allotment	Specific Tasks & Deliverables

Person A	Data Engineer & Test Harness	60 Mins	1. Dataset Creation: Build <code>equipment.csv</code> (10 items), <code>contractors.csv</code> (10 companies), and <code>inquiries.csv</code> (15 test cases) with realistic fields like <code>license_held</code>, <code>units_available</code>, and <code>additional_notes</code>. 2. Test Fixtures: Create 5 "Chat Input" strings that represent real-world contractor messages (e.g., <i>"Urgent! Need a JCB for tomorrow"</i>) to validate the parser. 3. Dynamic Logger: Write a Python function to append every agent decision to a <code>chat_log.csv</code> at runtime to prove "updated records".
----------	------------------------------	---------	--

Person B	Agentic Logic & NLP Core	120 Mins	1. The Parser: Write the <code>parse_inquiry(text)</code> function using Regex/Keywords to extract: <code>equipment_id</code>, <code>duration_days</code>, <code>start_date</code>, <code>site_risk</code> (tight/normal), and <code>urgency</code> (high/normal). 2. The Scorecard: Implement the <code>calculate_score(parsed_data, equipment, contractor)</code> function with weighted gates: - Availability (30 pts) - License Match (25 pts) - Lead Time / Urgency (25 pts) - Contractor Tier (20 pts). 3. The Decision Engine: Write the <code>run_rental_agent(user_input)</code> function that combines parsing + scoring + reasoning to return a structured response (Score, Decision,
----------	-----------------------------	----------	---

			Quote breakdown, or Missing Info request).
Person C	Frontend UI/UX Developer	90 Mins	1. Streamlit App Layout: Build the chat interface using <code>st.chat_input</code> and <code>st.chat_message</code>. 2. Display Rendering: Ensure the agent's output is formatted beautifully using Markdown (bold headers, bullet points, colored boxes for Auto-Quote/Review). 3. Session State Management: Implement <code>st.session_state</code> to maintain chat history across interactions so the judge can scroll through the entire reasoning thread.

Person D	Integration & QA Specialist	90 Mins	1. Pipeline Integration: Connect Person B's <code>run_rental_agent()</code> directly to Person C's <code>st.chat_input</code> without breaking. 2. Error Handling: Ensure the bot doesn't crash if the user types gibberish (e.g., if no equipment is found, the bot replies: <i>"Could you specify the machine?"</i>). 3. Demo Script: Write a 3-minute "Judge Walkthrough" script with 4 specific chat inputs (valid, invalid, urgent, missing license) to showcase dynamic reasoning. 4. Final Polish: Add a "Export Report" button that downloads the current chat log as a <code>.txt</code> file.
----------	-----------------------------	---------	---

3. Timelines & Critical Dependencies (The "Integration Window")

To avoid last-minute chaos, stick to this strict timeline:

Time (Hackathon Clock)	Action Items
Hour 0 - 0.5	All 4: Brainstorm UI wireframe and finalize CSV columns together.
Hour 0.5 - 1.5	Person A builds CSVs. Person B starts writing Regex parser. Person C builds the empty Streamlit skeleton. Person D writes the data-loading functions.
Hour 1.5 - 3.0	(CRITICAL): Person B finishes the scoring logic. Person A hands over CSVs. Person C waits for the logic to plug in. Person D begins integrating the frontend and backend.
Hour 3.0 - 4.0	All 4: Hard Integration Phase. Person C and D finalize the chat UI. Person B tweaks the reasoning text based on initial test inputs. Person A runs the 15 test cases to ensure the score threshold works.
Hour 4.0 - 5.0	Bug Fixing & Polish: Person D runs the demo script. Person C adds emojis and color-coded risk indicators. Team prepares final 3-minute pitch.

4. Shared Deliverables (Due at the Final Submission)

- GitHub/Project Repo: Contains all code and the `data/` folder.

- The `chat_log.csv`: Must show at least 10 different interactions where the dataset was "updated" (simulated by logging the new status of the equipment or inquiry).
 - Screenshots: Person C must take 4 screenshots of the chat window showing: *Auto-Quote*, *Request Info*, *Manual Review*, and *Unrecognized Input*.
 - Presentation Slides: A 5-slide pitch covering: The Problem, Our Agent's Reasoning Gates, The Chatbot Interface, A Live Demo Screenshot, and Future Scope.
-

5. Dynamic Reasoning Proof Plan (For Judges)

To guarantee a high score, we will ensure the agent shows Chain of Thought in every chat response:

" 🧠 *Agent Thought Process:*

1. *Parsed: Dozer, 5 days, Sep 16.*
2. *Stock Check: Available (30/30 pts).*
3. *License Check: Contractor holds Class A (25/25 pts).*
4. *Lead Time: 11 days out - Low Urgency (25/25 pts).*
5. *Tier: Gold Tier (20/20 pts).*

Total Score: 100/100 → Decision: Auto-Quote."